linope

> The road ahead is long and arduous; I shall seek high and low.

This technical report introduces the design philosophy, development paradigm, and technical details of the Linope kernel in a clear and accessible way.

Abstract

As more companies and developers begin building Agents, one question has become increasingly worth discussing:

What is the difference between the kernel of an Agent system and the kernel of a traditional operating system?

With this question in mind, I developed the Linope Agent kernel. Linope aims to draw on the design principles of traditional operating systems while incorporating the characteristics of Agent systems, exploring a development paradigm different from that of traditional Agent frameworks.

This article focuses on two aspects:

- Linope’ s design philosophy;
- Linope’ s Agent development paradigm.

I have often wondered: if a Coding Agent and a medical consultant Agent used the same kernel, where exactly would their differences lie?

If we replace the tools, modify some prompts and knowledge bases, and connect websites, software, or other services through MCP, could a Coding Agent become a medical consultant Agent?

Today, we can use Agents such as Codex and Claude Code to complete many different types of tasks. These Agents do not fundamentally change their internal coordination logic when the task changes from coding to searching, writing, or analysis. Developers usually only need to connect new tools, add new Skills, or adjust the knowledge sources to give an Agent new capabilities.

This is made possible by the continuous improvement of LLMs’ general capabilities. In the past, we often needed to use large amounts of code to control an Agent’ s workflow. Today, models are becoming increasingly capable of planning, understanding, and creating, so many complex tasks can be delegated to the model to decompose and advance on its own.

The open-sourcing of DeepSeek Harness further illustrates this point: an Agent can gain new capabilities through plugins without requiring a completely new Agent to be written for every task.

Therefore, Linope defines an Agent as:

> Agent = Kernel + Capabilities

In Linope, the kernel does not implement specific models or tools. Instead, it loads external capabilities into peer-level capability processes. A capability process may be an LLM, a search tool, a file-operation tool, or another service. The kernel should also provide a certain degree of flexibility.

Following this idea, an Agent OS kernel could borrow the design principles of traditional operating systems and manage processes, threads, and resources, including:

- Inter-process communication;
- Process lifecycles;
- Thread startup and concurrency control;
- Load balancing;
- Resource management;
- Context management;
- Network and sandbox management.

In an Agent system, these capabilities correspond to model calls, tool calls, context retrieval, communication between capability processes, and resource scheduling under highly concurrent workloads.

It is important to note that the capabilities inside the kernel are primarily responsible for coordinating external capability processes. They are not responsible for implementing every individual capability.

A New Agent Development Paradigm

As mentioned above, Linope’ s design philosophy is:

> Agent = Kernel + Capabilities

The new paradigm explored by Linope is not merely about placing models and tools into several independent processes. It also redefines the composition of an Agent.

An Agent is no longer a program filled with internal business workflows. Instead, it is a group of peer-level capability processes coordinated by the kernel.

In many Agent frameworks, an Agent is usually centered around a Session or an Agent instance. Model calls, tool calls, context assembly, compression, and task loops all revolve around this

instance. Developers add capabilities to the Agent by registering plugins, tools, and event listeners.

Taking `deepseek-harness` as an example, it uses Cordis to organize plugins, uses `SessionEvent` to preserve conversation facts, projects them into messages visible to the model through a surface, and finally uses an Agent Loop to drive turns, steps, model requests, and tool calls.

Its core relationship can be summarized as:

```
代码块
1  Plugin system
2      ↓
3  Agent Loop
4      ↓
5  SessionEvent log
6      ↓
7  Message projection
8      ↓
9  Model request
```

The advantage of this approach is that conversations are easy to persist, replay, and recover. Everything the model sees can be reconstructed from the Session log. During context compression, a summary message can replace an old range of messages while the original events are preserved.

However, the basic unit remains a complete Agent session. Tools, models, and context managers are usually attached to the session or runtime as plugins. They do not naturally possess independent resource ledgers, address spaces, or lifecycles.

Linope takes a different approach.

In Linope, the kernel does not treat models, tools, and context compressors as several functions inside an Agent. Instead, it treats them all as capability processes:

- A model process represents reasoning capability;
- A search process represents web-search capability;
- A code-execution process represents execution capability;
- A context-compression process represents context-processing capability.

None of them naturally belongs to another. The kernel is responsible for:

- Creating and reclaiming capability processes;
- Managing the resources held by processes;

- Passing requests between capability processes;
- Selecting processes that are still alive and able to handle tasks;
- Managing resources such as memory, tools, sandboxes, and networks;
- Scheduling according to context pressure and process load.

Therefore, the main difference between Linope and `deepseek-harness` lies in where plugins and capabilities are positioned.

In `deepseek-harness`, plugins mainly extend an Agent runtime. Plugins register services, events, and capabilities through Cordis, while the Agent Loop drives the conversation forward.

In Linope, capabilities themselves are objects that can be scheduled by the kernel. Models and tools have independent process identities and maintain their own resource leases and ledgers. A model can inspect the currently live capability processes through a page provided by the kernel, and then decide which process to call.

This also explains why Linope removes the concepts of parent-child processes and process trees.

Removing parent-child processes does not mean that capability processes have no relationships. It means that these relationships are no longer determined by "who started whom." Relationships between processes are established through resources, addresses, and communication rather than inherited through a process tree.

From a development perspective, Linope changes the following:

In the past, developers usually designed an Agent workflow first, and then inserted model, tool, and context-processing code into that workflow.

In Linope, developers first declare the capabilities and resources required by the Agent, and then implement each capability as a process that can be started and called by the kernel. Whether the Agent should search, execute code, ask another model, or read historical context is decided by the model based on the current page. This also reflects the idea that "the kernel should be flexible."

As models become more capable, we should delegate more decisions to them. The kernel should assist the model by providing coordination and scheduling capabilities, rather than restricting the model's reasoning.

Reference:

https://www.google.com/s2/favicons?domain=anthropic.com&sz=64

Anthropic: Effective context engineering for AI agents

This does not mean that Linope has no workflows at all. Process lifecycles, permission checks, resource reclamation, communication, and request validation are still handled by the kernel. The change is that the kernel no longer hardcodes specific business workflows.

The kernel ensures safe execution and controlled resources, while the model selects and plans among capabilities.

For example, a Coding Agent and a medical consultant Agent can use the same process, resource, and scheduling mechanisms. Their differences mainly come from:

- The model they use;
- The tools they mount;
- Their knowledge sources;
- Their system prompts and Skills;
- The contents of their memory;
- Their sandbox and network permissions;
- Their corresponding security policies.

Of course, a medical scenario cannot be implemented merely by replacing prompts. Medical knowledge sources, privacy protection, risk classification, human escalation, and result auditing should all be added to the system as independent capabilities or resources. However, these changes do not require redesigning the Agent scheduling kernel.

This is the new development paradigm proposed by Linope:

> Describe the capabilities an Agent requires through resource declarations, provide those capabilities through peer-level capability processes, and use the kernel to uniformly manage their lifecycles, communication, and permissions—rather than writing a separate coordination system for every type of Agent.

From this perspective, `deepseek-harness` places greater emphasis on reliably saving and reconstructing a single Agent session, while Linope places greater emphasis on decomposing an Agent into a group of capabilities that can be isolated, shared, and scheduled.

Neither replaces the other. They focus on different layers:

代码块

```
deepseek-harness:
Session log → Message projection → Agent Loop → Model request

Linope:
Capability processes → Resource leases → Pointer spaces → Kernel scheduling
```

The core novelty of Linope is not simply changing an Agent into several processes. It is changing the organizational unit of an Agent:

> From “an Agent with plugins” to “a collection of capability processes coordinated by the kernel.”

Why Treat Capability Scripts as Processes?

A capability refers to an external capability. Creating an external capability process means giving an Agent a particular capability.

For example:

- Creating a model process gives the Agent reasoning capability;
- Creating a search process gives the Agent web-search capability;
- Creating a file-tool process gives the Agent file read/write capability;
- Creating a context-compression process gives the Agent context-management capability.

I treat external capability scripts as processes because, in traditional operating systems, processes are the basic unit of resource isolation.

Parent-child processes in traditional operating systems usually serve several purposes:

1. When a child process starts, it can inherit the parent’s address space, file descriptors, current directory, and environment;
2. Process groups and sessions can organize multiple processes and send signals or stop tasks collectively;
3. A parent process can wait for a child process and collect its exit status, while the kernel cleans up the corresponding process-table entry.

These mechanisms are suitable for program execution in traditional operating systems. However, capability calls in Agent systems have different characteristics.

A model may call a search tool during execution. The search tool may then call a file tool, after which the model may start another model to handle a subtask. If all these capabilities are organized into a fixed parent-child process tree, the relationships between processes will be determined at startup, limiting the flexibility of the system.

Linope therefore removes parent-child processes and process trees, allowing capability processes to remain peers.

This design provides several benefits.

Independent Process Lifecycles

When a tool process crashes, the kernel only cleans up that tool process. Other processes are not affected.

The model process can continue running, and other tool processes do not need to exit. This better reflects how Agents work in practice: capability calls happen dynamically, and one failed

tool call should not automatically destroy the entire Agent.

Resource Isolation Does Not Spread Through Inheritance

When a peer-level process starts, it only receives information explicitly granted by the kernel, such as:

- Resource handles;
- Resource types;
- Call parameters;
- Configuration required by the current capability.

It does not automatically obtain resource entity identifiers belonging to other processes. It also does not inherit all resources simply because another process is its "parent."

For environment variables, a capability only receives the secrets it explicitly declares that it needs. If a model capability declares that it requires a particular API key, the kernel copies that key into the process environment. Other sensitive environment variables are not leaked through process inheritance.

Resource Sharing Must Be Explicitly Declared

Peer-level processes do not automatically share memory or other resources merely because they belong to the same Agent.

By default, memory in the same scope may also be compiled into different exclusive resource entities. Only an explicit sharing declaration causes processes to use the same resource entity.

In the current implementation, `--share`, the resource-sharing command-line option, applies to resource types that have already been declared, such as memory. A model itself is not independently registered as a resource type in `kernel/resource`. A model's identity is mainly passed through capability configuration and process settings.

Therefore, sharing a model capability process and sharing a memory resource must be understood separately. They cannot simply be treated as the same type of resource lease.

Inter-Process Calls Target Currently Live Processes

Relationships in a traditional process tree are usually determined at creation time. However, Agent tool calls happen dynamically.

A model may first call a search tool, then a code-execution tool, and then another model. Whether a tool exists, is idle, or is still alive may change during execution.

Therefore, a model process in Linope does not locate tools through fixed parent-child relationships. Instead, it inspects the kernel page to find currently available capability processes and then chooses a target.

The relationships between processes are determined by current state rather than by the structure created in a process tree.

Resource Ledgers Can Be Settled Per Process

Linope maintains an independent resource record for every PID.

Leases are recorded in the corresponding process ledger:

- A process receives leases when it starts;
- The process uses resources while running;
- Leases are released when the process exits;
- The resource entity is unloaded when its reference count reaches zero.

Threads are merely execution mechanisms inside a process and do not maintain independent resource ledgers.

If resources were managed according to a process tree, stopping one node would require deciding where the resource leases of the entire subtree should go. The resource ledger would become tied to whoever happened to start whom.

Linope instead binds resources directly to processes. Memory is the context resource used by a process, a tool is the tool connection used by a process, and a sandbox is the execution space used by a process. When the process exits, these authorizations are revoked.

Inter-Process Communication

If capability processes are peers, they cannot rely on parent-child relationships to automatically transfer data. They need to exchange tasks and results through communication mechanisms provided by the kernel.

Inter-process communication in Linope mainly addresses three questions:

- How does one process send a task to another process?
- How does one process obtain another process’ s result?
- How can the sender’ s resources and context be prevented from leaking to the receiver?

When a model calls a tool, the model process can send a task to the tool process. However, the tool process does not need to see the model process’ s complete memory. The receiver only receives the current task and the resources it owns.

After the tool finishes, the result is returned to the model process through a communication channel. The result may be written into the model’ s own memory or returned as part of the next page.

In `kernel/communication/channel.py`, Linope distinguishes between two types of communication.

`MemoryChannel` is mainly used to write to and read from a process's own memory. It is suitable for storing user input, model results, tool results, and internal process context.

`ServiceChannel` represents a service channel that has already been opened by the caller. It provides:

- `send`: send a message to a specified address;
- `publish`: publish a message to a topic;
- `receive`: receive messages sent to the current address;
- `close`: close the channel.

The kernel itself does not decide which network transport should be used, nor does it automatically create communication connections for all capability processes. The caller must first open a channel and then communicate through that channel.

This means that communication mechanisms and capability implementations are separate:

```
代码块
1  Capability process:
2  Responsible for processing tasks
3
4  Communication channel:
5  Responsible for transferring tasks and results
6
7  Kernel scheduler:
8  Responsible for selecting communication targets and checking access permissions
```

For example, a model process does not need to know how a search tool is implemented internally. It only needs to know that a capability process currently exists that can handle search tasks. The kernel finds that process and passes the request to it through a channel.

This design also leaves room for future remote communication. Currently, two capability processes may run on the same machine and communicate through local channels. In the future, one process could be deployed on another machine and communicate through HTTP, RPC, or another protocol.

It is important to note that inter-process communication is not the same as sharing memory. A process may send a task through communication, but the receiver does not automatically gain access to the sender's context address space. Task content, result content, and memory access permissions remain three separate concerns.

Threads: Execution Units Inside a Process

In Linope, a process is the basic unit of resource isolation and accounting, while a thread is the basic unit for executing tasks inside a process.

A capability process may receive multiple requests at the same time. For example, a model process may wait for multiple tool results simultaneously, and a search process may handle multiple search tasks concurrently.

If every request started a new process, startup costs would be high and resource management would become more complex. Therefore, Linope uses threads to process tasks concurrently inside the same capability process.

Threads mainly perform the following duties:

- Claim tasks from the process input channel;
- Execute one capability call;
- Wait for a model, tool, or network response;
- Submit results to the process output channel;
- Process multiple tasks within the process’ s concurrency limit.

The relationship between processes and threads can be understood as follows:

代码块

```
Process:
Owns resources, handles, ledgers, and lifecycle

Thread:
Uses resources already acquired by the process to execute tasks
```

Threads do not independently own memory, sandbox, or network leases. They do not maintain independent resource ledgers. Multiple threads inside a process share the resources already acquired by that process. This is also consistent with the traditional operating-system distinction between processes and threads.

If a thread fails, it usually only affects the current request. If the entire capability process exits, all threads inside that process stop, and the resources held by the process are released together.

Therefore:

> Threads are suitable for concurrency, while processes are suitable for isolation and lifecycle management.

Linope does not treat threads as resource-isolation boundaries or independent capabilities.

Treating an Agent as a Service

In addition to treating an Agent as a process, we can also view an Agent as a service.

Processes and services are not mutually exclusive concepts. They exist at different layers:

```
代码块
1  Process:
2  How the Agent runs inside the system
3
4  Service:
5  How other processes, Agents, or external applications access the Agent
```

An Agent can run locally as a capability process while presenting itself externally as a service. The process has a PID, resource leases, and runtime state. The service has an access address, calling protocol, capability description, and lifecycle interface.

Once an Agent is treated as a service, it is no longer merely a script passively waiting for a model call. It becomes a system component that can be discovered, called, and composed.

For example:

- A medical Agent can serve as a medical question-answering service called by other Agents;
- A Coding Agent can serve as a code-analysis and code-writing service called by a task scheduler;
- A search Agent can serve as a search service shared by multiple model processes.

A sufficiently general Agent might be composed of multiple Agent services and designed according to specific business requirements. In the future, we may split and refine different Agents according to business needs in the same way that we design microservices, creating an Agent-oriented version of microservices.

This direction is consistent with the development of several Agent projects on GitHub.

AgentScope Runtime exposes Agents as Agent-as-a-Service and provides streaming APIs, tool sandboxes, observability, and deployment modes for local environments, Kubernetes, and Serverless. Its focus is not to treat an Agent as a one-time function call, but to wrap the Agent runtime as an application that continuously provides services.

The A2A project further abstracts communication between Agents into an open protocol. A2A allows Agents from different frameworks, companies, and servers to discover capabilities, negotiate interaction methods, and collaborate through JSON-RPC over HTTP(S), streaming transmission, or asynchronous notifications.

A2A particularly emphasizes that an Agent’ s internal state can remain opaque. The caller does not need to know which model another Agent uses, what memory it has, or which tools it owns.

It only needs to learn from the Agent Card what capabilities the Agent provides, how to connect to it, and which interaction methods it supports.

This corresponds to Linope in several ways:

```
代码块
1  Linope process:
2  Capabilities, resources, handles, PID
3
4  Linope service:
5  Address, service capabilities, communication protocol, call relationships
6
7  A2A:
8  Agent Card, task protocol, collaboration between remote Agents
```

Of course, these projects do not implement exactly the same thing. AgentScope Runtime focuses more on production deployment, A2A focuses more on interoperability between Agents, and Linope focuses on processes, resources, and scheduling at the kernel layer.

However, together they demonstrate an important trend:

> Agents are gradually changing from programs that can only be called internally into services that can be discovered and called by other systems.

From this perspective, Linope capability processes can serve as the runtime foundation of services. A service may be provided by one capability process or jointly by multiple capability processes. The kernel manages the process lifecycle, resource leases, and load balancing behind the service, while the service interface exposes capabilities externally.

Communication Between Services

Inter-process communication mainly occurs between capability processes managed by the kernel. Service-to-service communication occurs between two independent services.

The difference is that service-to-service communication does not necessarily happen on the same machine or under the same process manager. Services may involve:

- Network latency;
- Service discovery;
- Identity authentication;
- Permission control;
- Request timeouts;
- Retries;

- Task-status queries;
- Streaming output;
- Asynchronous callbacks;
- Version compatibility.

Therefore, service-to-service communication cannot simply be treated as “moving inter-process communication onto a network.” It requires a clearer service contract.

At minimum, a service should specify:

- Who it is;
- What tasks it can complete;
- What input it accepts;
- What results it returns;
- Whether tasks can run for a long time;
- Whether streaming results are supported;
- How task status is queried;
- How tasks are cancelled;
- What permissions are required.

Linope’s current `ServiceChannel` already provides a direction for this abstraction. The caller communicates with a service through `send`, `publish`, and `receive`, while the actual transport—local channels, HTTP, RPC, or something else—can be implemented externally.

In this way, the kernel manages service scheduling and resources, while the communication layer manages data transfer between services.

HATEOAS and Hypermedia Links in Linope

How can a service tell its caller what can currently be done? This is an important question in service design.

A common approach is to hardcode all interfaces into the client:

代码块

```
The client knows:

/search
/tool
/ask
/start
```

```
7   /stop
```

The problem with this approach is that the client must know all operations in advance. If a service is currently unavailable, permissions change, or the service adds a new operation, the client may still issue requests according to the old interface.

Linope takes a different approach: it allows the current page to tell the model or caller which actions are currently available.

In `kernel/hypermedia`, a page consists of facts and links. A memory page may contain:

- The current memory state;
- The content of the current turn;
- Addresses of historical content;
- Readable memory links;
- Callable tools;
- Models that can be asked;
- Capability processes that can be started;
- An entry point for the final answer.

`memory_page()` generates memory-related links, while `model_page()` adds links for tools, models, and starting capabilities based on currently live processes and permissions.

This design is inspired by HATEOAS.

HATEOAS is a constraint of the REST architecture. Its full name is:

> Hypermedia as the Engine of Application State

Its core idea is that a client does not need to know all possible state transitions of a service in advance. Instead, it discovers the next available actions through hypermedia links returned by the service.

The main references are:

1. Roy Fielding’ s doctoral dissertation, especially the definition of REST and the constraint “Hypermedia as the engine of application state”：

https://www.google.com/s2/favicons?domain=uci.edu&sz=64

1. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
2. RFC 8288, “Web Linking,” which defines web links, link relations, and target resources:

https://www.google.com/s2/favicons?domain=rfc-editor.org&sz=64

1. https://www.rfc-editor.org/rfc/rfc8288

It should be noted that Linope’ s current `kernel/hypermedia` is not a complete REST service implementation, nor is it simply a matter of giving web links to a model to click. What Linope borrows is the core idea of HATEOAS: dynamically provide executable relationships and entry points according to the state of the current resource.

For example, a model page might tell the model:

代码块

```
memory/message/0
tool:search
ask:model-02
process start tools.ddgs
answer
```

The model does not need to remember every tool command in advance. It only needs to select an available link from the current page.

Linope uses this design for several reasons.

First, It Reduces Hardcoded Scheduling Logic

If models and clients hardcode every tool and process in advance, adding a new capability requires modifying the model prompt, client code, or scheduling logic.

When the page provides links, new capabilities can be displayed dynamically by the kernel. The model only needs to understand the current page rather than knowing every capability that may exist.

Second, It Reflects the Current Runtime State

A tool process may have exited, a model may be busy, a capability may not have been granted, or a service may be unavailable due to permissions.

These states can directly determine whether the corresponding link appears on the page. The page displays not all capabilities in the system, but the capabilities that can currently be attempted.

Third, It Separates Discovery from Execution

A link on a page is only an executable entry point, not the final authorization.

Even if the model sees an address, the scheduler must still check:

- Whether the process exists;
- Whether the resource exists;
- Whether the current process has the corresponding permission;

- Whether the selector is inside the pointer space;
- Whether the handle is still valid.

Therefore, an HATEOAS-style page tells the model “what can be attempted,” while the kernel permission system decides “what is actually allowed.”

Fourth, It Is Suitable for Service-Oriented Extensions

In local process mode, a link may point to a capability process, a memory address, or a kernel command.

In a future service mode, a link may also point to a remote service, task resource, or A2A operation.

For example, a remote medical Agent might return:

代码块

```
self
capabilities
task
status
cancel
result
```

The caller does not need to know how many processes are inside the service. It only needs to follow the link relations to complete the task. This creates a degree of decoupling between the client and the service.

Fifth, It Reduces Context Waste

Traditional approaches may place all tool descriptions, historical content, and available commands into the model context.

Linope’s page only provides addresses and actions relevant to the current situation. When the model needs historical information, it reads the corresponding memory address. When it needs a tool, it follows the relevant tool relation.

This is consistent with an important principle of context engineering:

> Do not put all potentially useful information into the context at once. Provide information relevant to the current decision when it is needed.

It is important to emphasize again that links are not security authorizations. A model may fabricate an address string, but that does not grant it access to a resource.

All actual reads and calls must still pass selector checks, pointer-space checks, and resource-handle checks inside the Linope kernel.

Agent Services and Future Microservice Experiments

As mentioned earlier, a sufficiently general Agent might be composed of multiple Agent services and designed according to specific business requirements.

Currently, Linope mainly operates through local processes and kernel scheduling. In the future, a capability process could be wrapped as an independent service with:

- An independent service address;
- An independent version;
- An independent resource quota;
- An independent deployment location;
- Independent observability;
- An independent upgrade cycle.

At that point, the Agent call relationship could evolve from:

```
代码块
1   Model process → Kernel → Tool process
```

into:

```
代码块
1   Model service → Service discovery → Tool service
```

The kernel could still manage resources and permissions, while communication could use HTTP, RPC, A2A, or another protocol.

However, turning Agents into services does not simply mean adding an HTTP interface to an existing process. True microservice adoption requires addressing many additional issues:

- How should tasks recover after a network interruption?
- Will duplicate requests cause repeated side effects?
- How should service versions remain compatible?
- How should long-running tasks be queried?
- How should remote services be authenticated?
- Which side owns the memory?
- Should tool results be persisted?

- Who is responsible for retries when a service fails?
- How should trace and audit information be passed between services?

Therefore, the more accurate statement is:

> Linope decomposes an Agent into peer-level capability processes and preserves the possibility of evolving toward Agent microservices through resources, communication, service descriptions, and hypermedia links.

A process is the local runtime carrier of a service. A service is the external access boundary of a capability.

Both can exist at the same time:

```
代码块
1   Agent service
2       ↓
3   One or more capability processes
4       ↓
5   Resource leases, threads, tools, and memory
```

In the future, changing the communication method will not require redesigning the capability itself. It will only require adding new service providers and transport implementations for that capability.

Linope’ s Resource Design

Resource management is an important responsibility of an operating-system kernel.

Traditional operating-system resources usually include:

- CPU;
- Memory;
- I/O devices;
- Disks;
- Network devices.

However, Linope runs above a traditional operating system. The CPU, GPU, physical computer memory, and disks are still managed by the host operating system. Linope uses these resources through the processes, files, networks, and system calls provided by the host.

This is consistent with the principle of not reinventing the wheel.

The Linope kernel focuses on Agent-specific resources:

- Memory;

- Tools;
- Sandboxes;
- Networks.

These are not hardware resources. They are capability resources that an Agent needs to request, isolate, share, and reclaim at runtime.

The basic relationship of resource management in Linope can be summarized as:

```
代码块
1   Requirement → Declaration → Entity → Lease → Handle
```

A capability first declares which resources it needs. Based on the declaration, the kernel creates or obtains a resource entity, establishes a lease for the specific process, and issues a handle to that process.

What the process actually sees is the handle, resource type, and call parameters—not the complete internal structure of the kernel resource.

Resource entities and process ledgers are also separated:

- A resource entity stores the actual runtime state;
- A lease manages who is using the resource;
- A process ledger records which resources a PID holds;
- A handle is used to check whether the caller still has access.

This design means that the kernel does not need to modify process-management logic for every new resource type. As long as a new resource type is registered in the resource contract, the existing entity, lease, handle, and ledger mechanisms can be reused.

Memory: The Context Space of an Agent

If tokens are treated as the units of memory in an Agent OS, then a model’ s context window is similar to the amount of available memory in a traditional operating system.

The amount of content that can be carried in a model call depends on the maximum context length supported by the model. The longer the context, the more historical information the model can see. Once the context is full, it must be trimmed, compressed, or delegated to another process.

However, tokens are not the same as Linope memory.

More precisely:

- Memory stores context content;

- Context is projected from memory and prepared to be sent to the model;
- Tokens measure how much context space the content occupies;
- The context limit is the maximum context length the model can accept.

Therefore, Linope memory is closer to “context memory” than to a database, vector database, or permanent memory system.

Comparison with Session-Log-Centered Agent Frameworks

Taking `deepseek-harness` as an example again, it stores an Agent session as an append-only `SessionEvent` log. User messages, model messages, and tool results all become events.

The system then uses a surface to project events into the model context as a list of messages, after which `deriveMessages()` reconstructs the model history.

Its core relationship is:

代码块

```
SessionEvent log
    ↓
Surface projection
    ↓
deriveMessages()
    ↓
Model message history
```

During context compression, the system replaces an original range of messages with a summary message. The original events remain in the log, while the model sees the replacement in the next surface.

This approach emphasizes:

- Persistence of the session log;
- Replayability of requests;
- Traceability of compression;
- Deterministic reconstruction of the model context.

Linope focuses on a different problem.

In Linope, memory is not merely another form of session log. It is a kernel resource with a resource identity, access permissions, virtual addresses, and lifecycle.

The two approaches can be summarized as follows:

代码块

```
Session-log-centered:
Event log → Message projection → Complete context → Compression replacement

Linope:
Resource lease → Pointer space → Selector authorization → On-demand reading
```

The former solves the problem of reliably saving and reconstructing an Agent session. The latter solves the problem of how multiple peer-level capability processes can isolate, share, and access context.

Memory Is a Resource

In a traditional operating system, a process does not directly own the physical memory of the entire computer. Instead, it obtains a managed address space from the operating system.

Linope follows a similar approach.

Memory is registered as a resource type, and the caller must provide a scope, such as:

代码块

```
memory=chat
memory=medical-session
memory=coding-task
```

The scope is the name of a logical memory space. Based on the resource type and identity, the kernel generates a `resource_id`, and then creates a lease and handle for the process.

A model does not automatically own memory. In the current project, a model capability declares a network resource by default. A memory resource can be attached when needed using:

代码块

```
--with memory=chat
```

With this design:

- A model can have no memory and accept only one-time input;
- A model can have an independent memory;
- Multiple models can explicitly share the same memory;
- A context-compression capability can also use memory as an independent process.

The kernel does not care whether the memory stores a coding conversation, a medical consultation, or a search task. It only creates the resource, assigns permissions, and manages its

lifecycle.

Pointers: Entry Points for Context Access

Linope memory is not a dictionary that a process can read directly. A process must first obtain a pointer to the memory.

In `kernel/scheduler/memory_space.py`, a memory pointer contains:

```
代码块
1   scope       Name of the memory
2   resource_id Underlying resource entity
3   classes     Permitted categories
4   handle      Handle held by the process
```

Therefore, a pointer is not merely an address. It also contains an access scope.

For example:

```
代码块
1   scope = chat
2   classes = ("message", "artifact")
```

This means that the current process can access messages and tool results in `chat`, but cannot access tasks or system content.

The kernel builds each process’ s pointer space according to the resources already granted to that process.

Even if a process knows the name of a memory, it cannot access it directly. Access succeeds only when all of the following conditions are satisfied:

1. The process holds the corresponding memory;
2. The scope exists in its pointer space;
3. The target category is within the permitted range;
4. The resource handle is still valid.

Memory Stores the Original Content

When a new memory is opened, it initially contains only address-space information:

```
代码块
1   address_space:
```

```
2       scope: chat
```

As the process works, the memory gradually forms a dictionary containing:

- `items` : the actual messages, tasks, and tool results;
- `next` : the next number for each category;
- `seq` : the global incrementing sequence number of the entire memory;
- `system` : the system prompt;
- `prompt_files` : prompt files that have already been loaded.

An individual record usually contains:

- `class` : whether it belongs to `message` , `task` , `artifact` , or `system` ;
- `kind` : whether it is `user` , `model` , `tool` , or `other` ;
- `content` : the original content;
- `time` : the write timestamp;
- `seq` : its order in the entire memory;
- `state` : such as `asked` , `resulted` , `rejected` , or `answered` .

Different types of content are placed into different categories:

- User input and model output belong to `message` ;
- Tool-call results belong to `artifact` ;
- Other scheduling tasks belong to `task` ;
- System prompts belong to `system` .

For example:

代码块

```
1   message/0   → User question
2   message/1   → Model response
3   artifact/0 → Tool result
4   task/0      → Task assigned to another process
```

The identifiers are not global. `message/0` and `artifact/0` can exist at the same time because they belong to different categories.

The `seq` field is used to order records across categories. Therefore, memory can be accessed by category or reorganized into model messages according to chronological order.

System prompts are also not simply concatenated into every request. Prompt files written through `write_prompt` are stored in `prompt_files` and combined into a single system entry according to their specified positions.

Prompt files with the same name are replaced rather than appended indefinitely.

Why Does Each Process Need Its Own Address Table?

Multiple processes may share the same memory entity, but they should not therefore share the same local addresses.

For example, process A first adds a message to memory:

```
代码块
1  Underlying memory:
2  message/0 = First message
3
4  Process A:
5  message/0 → message/0
```

Later, process B joins, and process A writes another message:

```
代码块
1  Underlying memory:
2  message/0 = First message
3  message/1 = Second message
4
5  Process A:
6  message/0 → message/0
7  message/1 → message/1
8
9  Process B:
10 message/0 → message/1
```

At this point, A's `message/1` and B's `message/0` point to the same actual record.

Linope distinguishes between several types of addresses:

- `address`: the local address visible to the process;
- `real`: the name of the actual record inside memory;
- `scope`: the name of the resource space;
- `resource_id`: the internal identifier of the kernel entity.

This is similar to a virtual address space in a traditional operating system: multiple processes can map the same underlying resource, while the virtual addresses they use do not need to be the same.

Memory Has Its Own Address Space

If there were only one shared dictionary, multiple processes using `message/0`, `message/1`, and so on would cause problems.

Different processes may join the memory at different times and may have access to different categories. If every process directly used the underlying identifiers, a newly joined process would automatically see all previous records, breaking isolation.

Therefore, Linope divides memory into two layers:

1. The memory entity, which stores the actual content;
2. The process-specific pointer space, which determines which memory the process can enter and which categories it can see.

For example:

代码块

```
message/0
message/0-10
artifact/0
chat/message/0
```

Here:

- `message/0` is the first message in the current process’ s address space;
- `message/0-10` is the range of messages visible to the current process;
- `chat/message/0` means accessing the first message in the memory named `chat`;
- The scope itself is not a permission. Reading is allowed only when the process actually owns a pointer to `chat`.

How Does a Model Use Pointers?

In page mode, the model does not initially receive the complete message history.

`memory_page()` places the content of the current turn directly on the page, while older content is displayed only through its address, type, and timestamp:

代码块

```
message/0
```

```
artifact/0
message/1
```

If the model needs to inspect historical messages, it generates a `MemoryRead` request. After authorization, the actual content is read.

The process is roughly:

代码块

```
Model requests message/0-3
    ↓
authorize_selector()
    ↓
PointerSpace.require()
    ↓
memory_contents()
    ↓
Generate the next page
```

An address on a page is only an entry point that “may be attempted.” It does not itself constitute authorization. Actual authorization remains under the control of the scheduler.

This on-demand reading model reflects an important principle:

> More context is not always better. The model should receive the information useful for its current decision.

Historical content does not automatically fill the context. The model chooses which addresses it needs based on the current task, and the kernel then checks whether it is authorized to read them.

A single model turn generally proceeds as follows:

1. The user’s question is written to memory;
2. The scheduler generates the current page;
3. The model chooses whether to read historical content, call a tool, ask another model, or answer directly;
4. The scheduler validates the model’s request;
5. The approved action is executed;
6. The tool result or result from another process is written back to memory;
7. The next page is generated;
8. The process continues until the model provides a final answer.

Reading memory does not directly start a tool or process. Reading is still only reading. The scheduler executes an action only when the model explicitly requests a tool call, starts a capability process, or asks another model.

Context Compression Is a Capability, Not Memory Itself

When compression is triggered and whether the original context is replaced after successful compression are policies determined by the host.

This reflects a fundamental design choice in Linope:

> The kernel manages context resources and access boundaries, while the compression capability decides how to rewrite the context.

The kernel should not decide for a particular Agent which memories are worth preserving.

For a Coding Agent, file paths, compiler errors, and test results may be the most important information. For a medical consultant Agent, medical history, medications, and temporal order may be more important.

Compression rules should belong to a capability rather than being hardcoded into the kernel.

Memory Lifecycle

The lifecycle of memory follows resource leases rather than a fixed parent-child process tree.

When a process starts:

1. It declares that it needs memory;
2. The kernel compiles a resource entity according to the scope;
3. The resource pool creates or obtains the memory;
4. A handle is assigned to the process;
5. A resource ledger and address space are established;
6. The process begins reading and writing content.

When a process exits:

1. The handle is released;
2. The memory reference count decreases;
3. If other processes still share the memory, it remains available;
4. If the reference count reaches zero, the resource is unloaded.

The current project’ s ledger stores only resource-accounting information, such as:

- PID;
- Resource type;

- `resource_id`;
- Handle;
- Identity;
- Lease status.

It does not store the actual message body.

In other words, the content of memory belongs to the runtime resource rather than being permanent data in the ledger file. Linope’ s current memory is more accurately described as kernel-managed runtime context memory, not a database for long-term knowledge storage.

If persistent user profiles, project knowledge, or medical records are needed in the future, databases, vector databases, or file systems can be introduced as new capability resources and exposed to Agents through tool processes.

Summary

The design of Linope can be summarized across several layers.

At the capability layer, models, tools, and context compressors are all peer-level capability processes.

At the process layer, every capability process has an independent lifecycle, PID, and resource ledger.

At the resource layer, the kernel manages memory, tools, sandboxes, and networks through resource entities, leases, handles, and reference counts.

At the context layer, memory stores original content, the pointer space manages access permissions, selectors describe read ranges, and the page mechanism allows the model to retrieve historical content on demand.

At the scheduling layer, the kernel routes requests according to currently live processes, resource status, context pressure, and concurrency.

Linope focuses on different concerns from session-log-centered Agent frameworks. The latter primarily solve how sessions are persisted, projected, and reconstructed. Linope goes further by decomposing an Agent into a group of capability processes that can be isolated, shared, and scheduled by the kernel.

Therefore, Linope proposes not merely a new Agent framework, but a new Agent development paradigm:

> An Agent is no longer a monolithic session bound to many plugins. It is a group of capability processes managed by the kernel. The model is responsible for planning and selecting capabilities, while the kernel is responsible for processes, resources, permissions, and communication.